

Homework 05

Question 1: Estimate the bias and variance of KNN

Solution 1(a)

The true mean response is

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 \approx 0.594218.$$

The simulation uses seed **43205**, 400 training observations, and noise standard deviation 0.1. Each KNN prediction averages the responses of the k nearest observations to $\mathbf{x}_0$. The predictions from one training dataset are:

Seed: 43205; true mean: 0.594218

k	KNN prediction
1	0.900898
3	0.637103
5	0.655623
7	0.655160
9	0.660551
11	0.621514
13	0.531781
15	0.505157
17	0.546222
19	0.517823
21	0.507187
23	0.521408
25	0.481105
27	0.473678
29	0.473241

Solution 1(b)

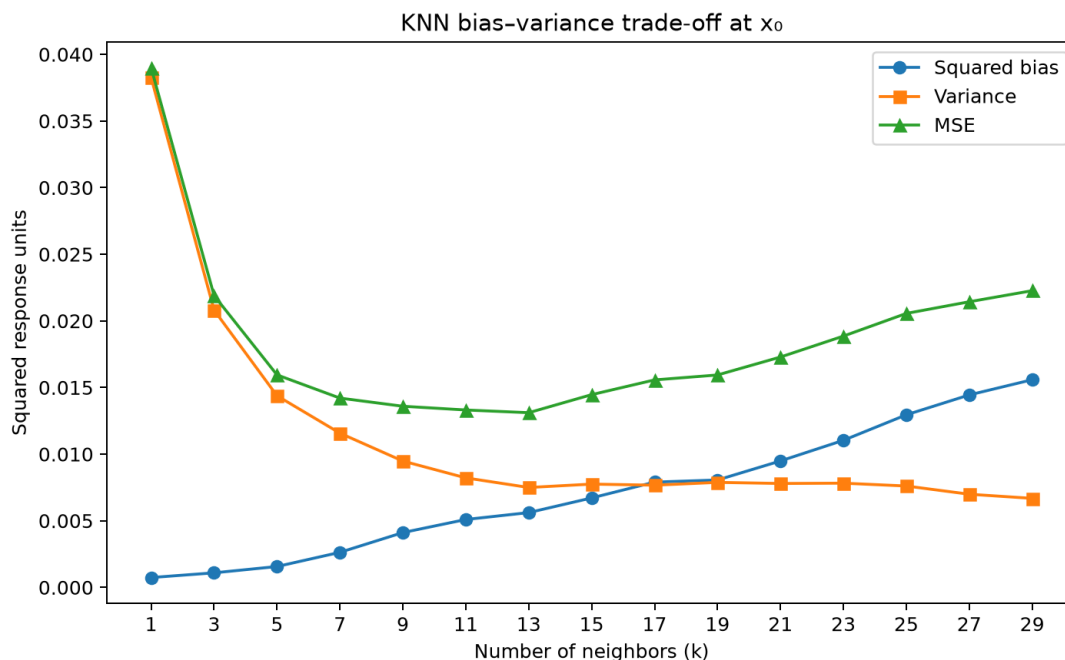
The 200 repetitions use independent training datasets, with the same dataset used for every k within a repetition. Let P_{rk} be the prediction in repetition r and $\bar{P}_k$ its average across repetitions. The empirical bias, variance, and MSE are

$$\widehat{\text{Bias}}_k = \bar{P}_k - f(\mathbf{x}_0), \quad \widehat{\text{Var}}_k = \frac{1}{200} \sum_{r=1}^{200} (P_{rk} - \bar{P}_k)^2,$$

$$\widehat{\text{MSE}}_k = \frac{1}{200} \sum_{r=1}^{200} (P_{rk} - f(\mathbf{x}_0))^2.$$

k	Bias	Squared bias	Variance	MSE
1	-0.027184	0.000739	0.038252	0.038991
3	-0.033052	0.001092	0.020807	0.021899
5	-0.039577	0.001566	0.014380	0.015946
7	-0.051288	0.002630	0.011574	0.014204
9	-0.064130	0.004113	0.009479	0.013592
11	-0.071374	0.005094	0.008218	0.013312
13	-0.074918	0.005613	0.007496	0.013109
15	-0.081932	0.006713	0.007750	0.014463
17	-0.088879	0.007899	0.007670	0.015569
19	-0.089777	0.008060	0.007879	0.015939
21	-0.097404	0.009488	0.007798	0.017285
23	-0.105048	0.011035	0.007820	0.018855
25	-0.113798	0.012950	0.007607	0.020557
27	-0.120174	0.014442	0.006991	0.021433
29	-0.124833	0.015583	0.006681	0.022264

Prediction matrix shape: (200, 15)
Maximum decomposition discrepancy: 4.337e-17
Smallest simulated MSE: 0.013109 at k = 13



For this simulation, the maximum absolute difference between MSE and squared bias plus variance is approximately 4.34×10^{-17} , which is floating-point rounding error. The identity therefore holds numerically.

Variance decreases overall as k increases, from about 0.038252 at $k = 1$ to 0.006681 at $k = 29$, although the estimated curve is not strictly decreasing. Averaging more responses makes the prediction less sensitive to individual training observations. Squared bias increases from about 0.000739 to 0.015583: larger neighborhoods include observations farther from the target, whose mean responses can differ from $f(\mathbf{x}_0)$. The MSE initially falls as the reduction in variance outweighs the increase in squared bias, then rises as squared bias becomes more influential. Among the tested values, the smallest simulated MSE is approximately **0.013109** at $k = 13$. This is a result of this finite simulation at this target point, rather than a universal optimal choice of k .

Increasing k in KNN is similar to increasing the penalty strength in ridge regression or lasso: it makes the fitted model less flexible, generally reducing variance at the cost of increased bias. The mechanisms differ. KNN averages over a larger neighborhood, ridge shrinks coefficients, and lasso can also set coefficients to zero.

In each case, an intermediate amount of smoothing or regularization can yield a better balance than either extreme.

The MSE here measures estimation of the fixed mean $f(\mathbf{x}_0)$, rather than prediction of a new noisy response. Therefore, the decomposition contains squared bias and variance, without adding another noise-variance term.

Question 2: Compare KNN with lasso

Solution 2(a)

With seed **43206**, each setting has 400 training and 1,000 test observations. Both methods use the same data. The 5NN fit uses all 30 covariates on their original scale and gives each neighbor equal weight.

Lasso uses ten-fold cross-validation over the specified 41 penalties. Predictor means and scales and the response mean are estimated from each fold's training portion only; scales use the population-variance divisor. Validation predictions reuse those quantities. The penalty with the lowest average validation MSE is selected, then the model and transformations are refitted on all training observations. The intercept is unpenalized, and Lasso's α matches λ in the stated objective.

The results for seed 43206 are:

Setting	5NN test MSE	Lasso test MSE	Selected lasso penalty	Nonzero lasso slopes
Independent covariates	0.555357	0.240285	0.050119	4
Correlated covariates	0.181553	0.240379	0.025119	5

Solution 2(b)

With independent covariates, lasso has a lower test MSE than 5NN. Only the first three covariates enter the response model, but KNN includes all 30 in its Euclidean distance. Differences in the other 27 covariates can make observations appear close or far apart without providing information about the response. As a result, the nearest neighbors in the full predictor space need not be particularly close on the relevant covariates.

In the correlated setting, the 5NN test MSE drops from approximately 0.555357 to 0.181553 and is lower than lasso's MSE. Positive correlation makes the covariates share information, so the additional covariates are no longer independent of the relevant ones. For this covariance matrix, variation is concentrated more strongly along a common direction. Euclidean neighborhoods can therefore become more informative about the relevant predictors, although variation in other directions remains. This provides an explanation for the improvement observed in this simulation.

Lasso's test MSE is nearly unchanged at about 0.240 in the two settings. Unlike KNN, lasso estimates predictor-specific coefficients and shrinks many to zero; it retains 4 slopes in the independent setting and 5 in the correlated setting. A predictor with a zero slope contributes nothing to its prediction, whereas every predictor contributes to KNN's distance calculation. With correlated predictors, lasso may select a predictor that serves as a substitute for another. The selected variables therefore should not automatically be interpreted as exactly the variables appearing in the true response model.

Solution 2(c)

The lasso model is restricted to an intercept and linear terms, but the true conditional mean includes both $\sin(X_2)$ and X_3^2 . A linear term may approximate some of the sine pattern over a limited range, but it cannot

represent the full nonlinear relationship. Similarly, an intercept can capture an average contribution from the squared term, but linear slopes cannot reproduce its curvature. Thus, lasso has model misspecification bias even if its penalty is well chosen. Shrinkage and variable selection do not remove this limitation.

KNN does not impose a linear response function and can adapt to nonlinear patterns when nearby observations provide useful information. However, it is sensitive to the geometry and dimension of the predictors. This comparison therefore reflects both lasso's restricted model form and KNN's difficulty finding useful neighbors; it does not isolate variable selection alone.

These results do not establish that correlation always helps KNN or that either method is always better. The observed ranking reverses between these two settings, and each setting uses only one generated training/test pair. Different correlation structures, response functions, sample sizes, noise levels, or random samples could change the results. Also, the comparison uses a fixed $k = 5$ for KNN while tuning lasso's penalty by cross-validation, so it applies specifically to the fitting procedures requested here.

Question 3: Observed dimension and latent dimension

Solution 3(a)

The simulation uses seed **43207**. Each dataset shares one loading matrix **A** across its 200 training and 200 test observations. KNN uses only the 100 observed covariates on their generated scale, with equal weights and Euclidean distance. All values of k use the same observations. The single-dataset test errors are:

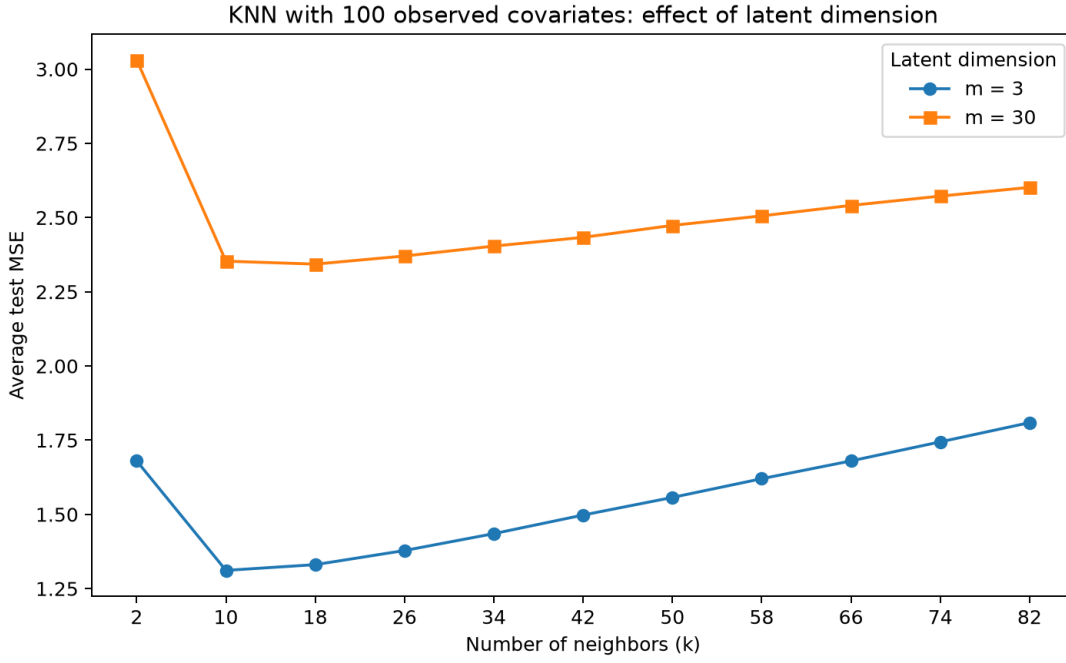
Seed: 43207

k	Test MSE (m = 3)	Test MSE (m = 30)
2	1.491480	3.189274
10	1.114946	2.529843
18	1.126857	2.428219
26	1.195094	2.510204
34	1.251545	2.501770
42	1.306968	2.536406
50	1.347847	2.580619
58	1.408301	2.600725
66	1.457312	2.647356
74	1.519822	2.691472
82	1.581855	2.727928

Solution 3(b)

The complete experiment is repeated 50 times for each latent dimension, with a new loading matrix and new observations in every repetition. The average test errors are:

k	Average test MSE (m = 3)	Average test MSE (m = 30)
2	1.680276	3.031204
10	1.311376	2.353107
18	1.330736	2.343187
26	1.378289	2.370534
34	1.435151	2.403928
42	1.497844	2.433288
50	1.557145	2.473599
58	1.620274	2.505676
66	1.680196	2.540958
74	1.744378	2.572255
82	1.809108	2.601737



The average test MSE is lower for $m = 3$ at every tested value of k . Its smallest average is approximately **1.311376** at $k = 10$, compared with **2.343187** at $k = 18$ for $m = 30$. Both curves initially decrease as averaging more neighbors reduces sensitivity to individual responses, then increase as larger neighborhoods smooth over observations with different response means. The advantage of the lower latent dimension persists across the tested neighborhood sizes.

These curves compare the specified choices of k in a simulation with known data generation. They are not cross-validation curves and are not used to select a final model using its test set.

Solution 3(c)

The number of measured covariates alone does not determine how informative KNN neighborhoods are. Conditional on the loading matrix, the signal $\mathbf{A}\mathbf{Z}_i$ lies in a subspace of dimension at most m . With $m = 3$, only three latent directions generate this signal, and two of those latent variables determine the response mean. With $m = 30$, there are 28 additional latent variables that affect distances without entering the response. Observations that are close in the full observed space may therefore still differ on the two response-relevant latent variables. With only 200 training observations, obtaining close neighbors across many latent directions is harder.

The measurement noise η_i perturbs all 100 observed coordinates and can obscure similarity in the latent signal. Without it, observations would lie in the column space of $\mathbf{A}$. With independent standard normal measurement noise, the conditional covariance is

$$\text{Cov}(\mathbf{X}_i | \mathbf{A}) = \mathbf{A}\mathbf{A}^\top + \mathbf{I}_{100}.$$

The identity term makes this covariance full rank, so the observed data are not confined exactly to an m -dimensional linear space. They have variation in every observed direction, even though their signal has lower-dimensional structure. The response noise ϵ_i is distinct from this measurement noise: its variance is 1, so predicting a fresh noisy test response has an irreducible error contribution of 1 even if the relevant latent variables were known.

Question 4: Classify handwritten digits

Solution 4(a)

The data consist of the first 1,000 training images and first 500 test images from the course CSV files. Distances use the 256 original pixel values, excluding the digit label, without standardization.

The function below calculates distances directly. A stable sort preserves training row order when distances tie. Counting votes in digit order and taking the first maximum breaks voting ties in favor of the smallest digit.

```
def classify_5nn(x_train, y_train, x_test, distance="euclidean"):
    if distance not in ("euclidean", "manhattan"):
        raise ValueError("distance must be 'euclidean' or 'manhattan'")
    predictions = np.empty(len(x_test), dtype=int)
    for i, image in enumerate(x_test):
        differences = x_train - image
        if distance == "euclidean":
            distances = np.sqrt(np.sum(differences ** 2, axis=1))
        else:
            distances = np.sum(np.abs(differences), axis=1)
        nearest = np.argsort(distances, kind="stable")[:5]
        votes = np.bincount(y_train[nearest], minlength=10)
        predictions[i] = np.argmax(votes)
    return predictions
```

Solution 4(b)

Euclidean: rows = predicted, columns = true

True	0	1	2	3	4	5	6	7	8	9
Predicted										
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

Manhattan: rows = predicted, columns = true

True	0	1	2	3	4	5	6	7	8	9
Predicted										
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

Classification errors:

Distance	Incorrect	Test classification error
euclidean	61	0.122
manhattan	69	0.138

Most confused pairs for euclidean (both directions):

Digit 1	Digit 2	Count
0	2	7
3	5	5
4	9	5
7	9	5
0	5	4

Most confused pairs for manhattan (both directions):

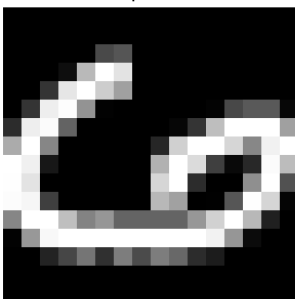
Digit 1	Digit 2	Count
0	2	6
7	9	6
0	5	5
3	5	5
4	9	5

The Euclidean classifier makes **61 errors out of 500 (12.2%)**, while the Manhattan classifier makes **69 errors (13.8%)**. For Euclidean distance, the most frequently confused pair is 0 and 2, with seven errors, all true 2s predicted as 0s. The pairs 3/5, 4/9, and 7/9 each account for five errors when both directions are combined. For Manhattan distance, 0/2 and 7/9 tie for the largest pairwise confusion count at six each. These are raw counts, which also depend on how frequently each digit appears in this test subset.

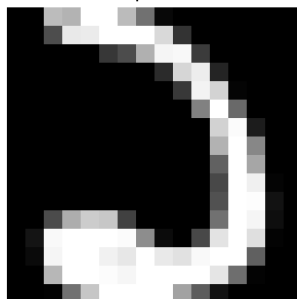
Solution 4(c)

Outcome	Number of images
Both correct	428
Only Euclidean wrong	3
Only Manhattan wrong	11
Both wrong	58
Different predicted labels	22

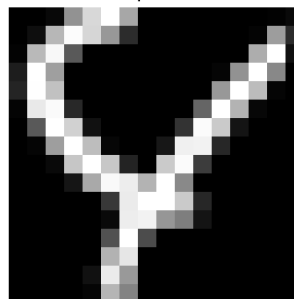
Test row 4
True: 6, predicted: 0



Test row 13
True: 2, predicted: 0



Test row 14
True: 4, predicted: 7



The first three Euclidean errors occur at test rows 4, 13, and 14:

- **True 6, predicted 0:** the broad curved outline and lower loop help explain why its pixel pattern resembles some zeros, even though the inward stroke suggests a six.
- **True 2, predicted 0:** the curved right edge and thick, curled base give this two a rounded appearance. Its open shape still distinguishes it visually from a typical zero, but pixel distance can emphasize the shared outline.

- **True 4, predicted 7:** this four has an unusual open, angled shape with a long sloping stroke. That makes the error somewhat understandable, although the branching strokes provide a visual clue that it is a four.

These examples illustrate that direct pixel distances do not explicitly recognize strokes or account for shifts and handwriting variations in the way a person might.

Euclidean distance performs slightly better on this subset, by 8 images or 1.6 percentage points. The two methods have similar overall error rates but do not make exactly the same mistakes: both are correct on 428 images, both are wrong on 58, only Euclidean is wrong on 3, and only Manhattan is wrong on 11. They predict different labels on 22 images; this exceeds the 14 images where only one is wrong because they sometimes choose different incorrect labels. Thus, similar total error can conceal differences in which images are misclassified. Euclidean distance emphasizes large individual pixel differences more strongly than Manhattan distance, which can change the nearest-neighbor ordering and final votes.

Question 5: Create a plot-formatting skill

Solution 5(a,b,c)

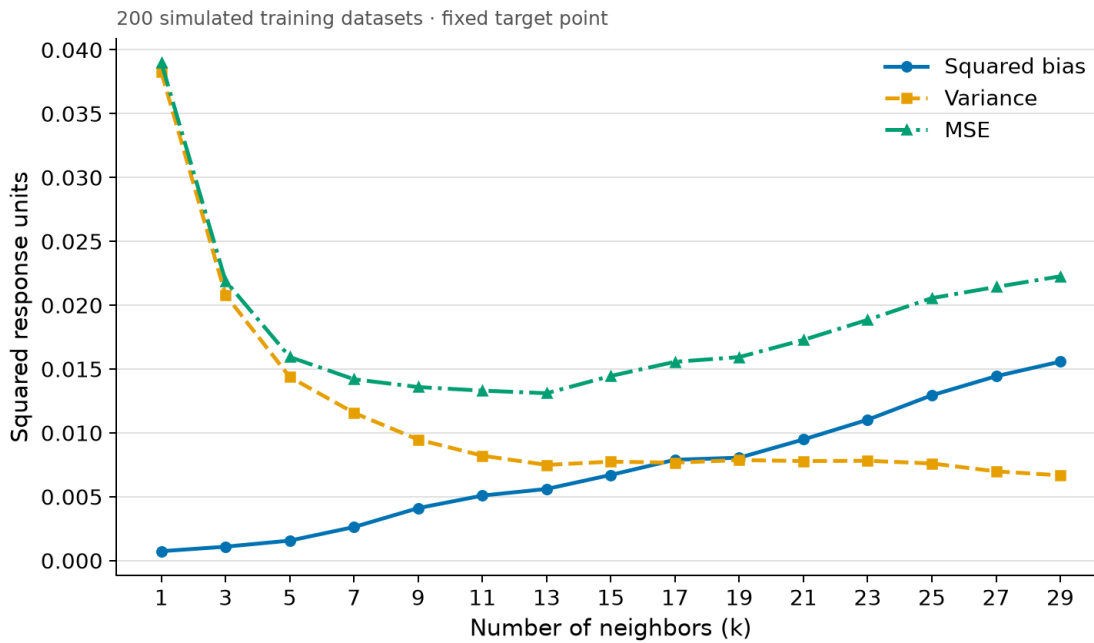
```
---
name: stat432-plot-formatting
description: Improve only STAT 432 homework plots in R or Python. Use this skill only when the user explicitly
  ↳ requests it; do not apply it automatically.
---
```

STAT 432 Homework Plot Formatting

- Preserve the data, calculations, and statistical meaning. Save the original plot for comparison.
- Margins: leave enough space for titles, tick labels, and legends; use constrained or tight layout in Python
↳ and suitable plot margins in R. Check the exported image for clipping.
- Titles: use a short descriptive title and, when useful, a subtitle explaining the sample or simulation
↳ context.
- Axes and labels: name both axes, include units when meaningful, use readable ticks, and keep limits
↳ consistent for before/after comparisons. Avoid misleading axis changes.
- Colors: use a colorblind-friendly palette such as blue (#0072B2), orange (#E69F00), and green (#009E73);
↳ also distinguish curves with markers or line styles.
- Sizes: aim for an 8-by-5-inch figure, 11-14 point text, and lines about 2 points wide. Adjust for the number
↳ of panels and export raster images at 150 dpi or higher.
- Keep grid lines subtle and legends clear of the data. Apply styling locally (for example, Matplotlib
↳ rc_context or a ggplot2 theme), then compare the original and revised plots and summarize the changes.

The skill was installed and explicitly applied to the Question 1 plot. Automatic invocation is disabled. The revised plot below uses the same values and axis limits as the original in Question 1.

KNN bias-variance trade-off



The revised plot uses a colorblind-friendly palette and different line styles as well as markers, making the curves easier to distinguish without relying on color alone. A subtitle gives the simulation context, light horizontal grid lines help compare values, and the simplified frame reduces visual clutter. Both versions use the same observations, calculated curves, and axis limits, so the formatting changes do not alter the conclusions.

The skill was tested once on the Question 1 plot. I prefer the revised plot because the grid lines make it easier to read the values of the points. The different line styles also help distinguish the curves, although the two versions are otherwise quite similar.

The rules cover common formatting choices in both R and Python, so the skill can be reused for other STAT 432 homework plots. Requiring an explicit request gives control over when it is applied. Testing it on the existing plot made the changes easy to compare without changing the results.